

PROJECT REPORT - RAYGAUSS PAPER REVIEW & IMPLEMENTATION



Nuages de Points et Modélisation 3D - course

March 2025

Nour **RIZK**
Matthieu **DESTRADE**

CONTENTS

1	Background	1
1.1	Neural Radiance Fields	1
1.2	3D Gaussian Splatting	2
2	RayGauss	3
2.1	Scene Representation with Gaussian primitives	3
2.2	Parameter Optimization	4
2.3	Slab-by-slab integration & leveraging BVH	4
2.4	Results	4
3	Our Approach	5
3.1	Basis functions	5
3.2	Color blending	6
4	Results	7
5	Conclusion	8
6	Appendix	10
6.1	Computation of derivatives for color blending	10

In this report, we present our project based on the paper *RayGauss: Volumetric Gaussian-Based Ray Casting for Photorealistic Novel View Synthesis*, H. Blanc et al.[1]. It presents a new method for photorealistic novel view synthesis (NVS) based on volumetric ray casting with Gaussian-based radiance field representation. First, we review the state of the art methods in novel view synthesis (NVS). Second, we provide a comprehensive overview of the paper’s method : RayGauss. Third, we present our implementation and results. We implement a rendering pipeline that is closer to Phong’s illumination [2] and that incorporates phenomena such as translucency. Doing so we improve the capabilities of the RayGauss approach. Finally, we provide insight on potential improvements and further applications.

1 BACKGROUND

Novel View Synthesis (NVS) consists in combining images from limited, known viewpoints of a scene to generate new viewpoints. This technique is essential in applications such as virtual reality, gaming and film production. Early methods relied on interpolation techniques or traditional 3D reconstruction, but recent advances in differentiable volumetric rendering and deep learning have led to highly photorealistic and efficient solutions. Among the most prominent approaches are Neural Radiance Fields (NeRF) [3] and Gaussian Splatting[4], each offering distinct trade-offs in quality, efficiency, and computational cost. In this section, we will briefly review those state of the art methods, before focusing on the RayGauss approach presented in the paper.

1.1 NEURAL RADIANCE FIELDS

First, Neural Radiance Fields (NeRF) has revolutionized NVS by introducing a neural representation of 3D scenes. In fact, instead of explicitly storing scene geometry, it optimizes a global neural network that implicitly models the entire scene by learning scene parameters : the radiance c and density σ values. Rendering is based on a ray casting algorithm, which considers a continuous radiance field. To compute the final pixel colour, rays

are sampled along their path and aggregated based on a differentiable volumetric rendering equation, as seen in Eq.(2). NeRF is able to capture view-dependent lighting effects such as specular reflections.

The rendering equation of NeRF, as well as 3D Gaussian Splatting, is derived from the radiative transfer equation in an absorbing and emitting material:

$$(\omega \cdot \nabla)L(x, \omega) = -\sigma(x)L(x, \omega) + \sigma(x)c(x, \omega) \quad (1)$$

where :

- $L(x, \omega)$ is the radiance field at position x in direction ω , which corresponds to the colour C of the ray in computer vision
- $\sigma(x)$ is the absorption coefficient, which corresponds to the density in computer vision
- $c(x, \omega)$ is the emitted radiance

In the paper, the authors provide the general solution of the classic volumetric rendering equation of Eq.(1). For NeRF, the differentiable volumetric ray casting algorithm corresponds to the computation of the discretized solution. For N samples along the ray, the discretized solution is given by :

$$C(r) = \sum_{i=0}^N (1 - \exp(-\sigma_i \Delta t)) c_i T_i \quad (2)$$

$$T_i = \exp \left(- \sum_{j=0}^{i-1} \sigma_j \Delta t \right)$$

where Δt is the discretization step, T_i is the transmittance for sample i . This solution assumes that the fields are piecewise constant in samples neighborhoods.

Although its compact scene representation makes it memory efficient, NeRF's computation is very costly. The approach requires extensive sampling and optimization, which makes training very slow. Additionally, it relies on a global scene parametrization, making it inefficient for local modeling of details. This led researchers to improve upon NeRF by developing models that can store local information of the 3D scene's parameters, as well as taking advantage of the sparse nature of some data structures to apply classic ray casting acceleration strategies.

In this perspective, RayGauss explores efficient ray casting computation on local basis functions (Gaussians) associated with points. This approach is different from the 3D Gaussian Splatting method, detailed below.

1.2 3D GAUSSIAN SPLATTING

Second, in contrast to NeRF's implicit representation, Gaussian Splatting implements an explicit point-based approach, where the scene is represented as a set of 3D Gaussian ellipsoids. This allows for efficient adaptation to the geometry of the scene. The Gaussians store color, opacity, and shape parameters and are projected onto the image plane, with rasterization based rendering. The explicit Gaussian-based representation coupled with the rasterization rendering, enables for real-time rendering at high quality (25-100 FPS). In fact, Gaussian Splatting was first introduced as a high-performance alternative to NeRF for real-time applications.

Additionally, training is more efficient than in the case of NeRF, since the approach aims at optimizing Gaussian parameters instead of a global neural network. This reduces the computation time and cost. However, the discrete representation of the radiance field as well as various limiting assumptions on the Gaussian structure induce an approximated graphical results and produce splatting artifacts such as flickering when changing the viewpoints. Thus, the authors decide to base their method on ray casting rendering, to suffer fewer approximations, given a high enough sampling rate.

Consequently, RayGauss addresses the two methods' limitations and provides a ray casting-based rendering of the radiance field which it decomposes into irregularly spaced elliptical and spherical basis functions, combining

the best attributes of both methods. In the following section, we detail the RayGauss approach, highlighting its physically consistent modelling of the density σ and emitted radiance c .

2 RAYGAUSS

In this section, we detail the RayGauss method presented in [1]. This approach combines the high quality rendering of volume ray casting with the flexible scene representation of Gaussian primitives.

2.1 SCENE REPRESENTATION WITH GAUSSIAN PRIMITIVES

RayGauss represents the scene as a radiance field decomposed into anisotropic 3D Gaussians, each with optimizable parameters (position, shape, orientation, density, and color). Unlike splatting-based methods, which approximate image formation by projecting Gaussians onto the image plane, RayGauss performs differentiable volume ray casting to achieve higher physical fidelity and avoid common splatting artifacts like flickering.

The rendering model is based on *Eq.(2)*, and depends on the formulation of functions c and σ . Thus, to preserve physical coherence and better adapt to the geometry of the scene without resolution limits, the authors have formulated these functions as a sum over their decomposition on irregular spatially distributed elliptical and spherical basis functions.

Formally, the authors assimilate the scene to N independent emissive and absorbing entities, distributed over the support of their basis function ϕ_l , with distinct density σ_l and emissive radiance c_l . For each entity l , its density at position x is given by :

$$\sigma_l(x) = \tilde{\sigma}_l \cdot \phi_l(x) \quad (3)$$

where $\tilde{\sigma}_l$ is the maximum density of entity l , and ϕ_l the basis function centered around the entity l at position x and quantifying the spread of the density ($\phi_l \in [0,1]$). The paper implements Gaussian basis functions $\mathcal{G}(x, \mu, q, s)$ as detailed below. However, since our experiments in Section 3 revolve around testing different basis functions, we construct the general framework before infusing the Gaussian basis functions in our explanation.

The authors then defined the radiance c_l of each entity as solely depending on the direction of light d , without taking into account the position x of the point. Thus, we have :

$$c_l(x, d) = c_l(d) \quad (4)$$

This formulation of the radiance allows for view-dependent effects, such as specular reflection.

Subsequently, to ensure physical coherence of the model parametrized by c and σ and consistency between the parameters c_l and σ_l , they apply the radiative transfer equation *Eq.1* per entity before summing over all entities, leading to the equation :

$$(d \cdot \nabla)L(x, d) = - \left(\sum_{l=1}^N \sigma_l(x) \right) L(x, d) + \sum_{l=1}^N \sigma_l(x) c_l(d) \quad (5)$$

which can be understood as characterizing the variation in radiance as the sum of an absorption and an emission phenomena caused by each independent entity.

Thus, the authors derive the total scene density σ and radiance c as given below :

$$\begin{aligned} \sigma(x) &= \sum_{l=1}^N \tilde{\sigma}_l \phi_l(x) \\ c(x, d) &= \frac{\sum_{l=1}^N c_l(d) \tilde{\sigma}_l \phi_l(x)}{\sum_{m=1}^N \tilde{\sigma}_m \phi_m(x)} \end{aligned} \quad (6)$$

The intuition being $c(x, d)$ is to understand it as the weighted average of the directional radiances from all entities at x , weighted by their local density contribution.

Being more flexible and a better match for real world detailed geometry, the authors implement anisotropic Gaussian functions as the basis class : $\mathcal{G}(x; \mu, \Sigma)$, with μ the mean and Σ the covariance matrix. Since the latter is a positive symmetric matrix, they decompose Σ into a rotation matrix R and a scaling matrix S according to the spectral theorem. Then, as commonly done in 3D modeling, they model the rotation matrix R with a quaternion $q \in \mathbb{R}^4$, and the scaling matrix S by three scaling coefficients such that $S = \text{diag}(s_x, s_y, s_z)$. Thus, the Gaussian basis functions is denoted as $\phi_l(x) = \mathcal{G}_l(x; \mu_l, q_l, s_l)$, and the expressions of the radiance and density are as follows :

$$\begin{aligned}\sigma(x) &= \sum_{i=1}^N \tilde{\sigma}_l \mathcal{G}_l(x; \mu_l, q_l, s_l) \\ c(x, d) &= \frac{1}{A} \sum_{l=1}^N c_l(d) \tilde{\sigma}_l \mathcal{G}_l(x; \mu_l, q_l, s_l)\end{aligned}\tag{7}$$

with $A = \sum_{m=1}^N \tilde{\sigma}_m \mathcal{G}_m(x; \mu_m, q_m, s_m)$ the normalizing factor.

Additionally, the authors use a combination of spherical harmonics (SH) and spherical Gaussians (SG) to model the radiance field. This approach improves the covering of both low frequency color representation (ambient and diffused light), which are better described by SH and high frequencies in charge of the specular effect that are better described by SG.

2.2 PARAMETER OPTIMIZATION

After detailing the scene parameterization, our goal is to optimize this set of parameters $P = \{(\tilde{\sigma}_l, \tilde{c}_l, \mu_l, q_l, s_l)\}_l$. As seen before $\tilde{\sigma}_l$ is the maximum density value, μ_l, q_l, s_l specify the geometry of the basis function, and $\tilde{c}_l$ defines the colormetric parameters of the basis function.

The objective function is given by :

$$L = (1 - \lambda)L_1 + \lambda \cdot \text{DSSIM}\tag{8}$$

with L_1 being the mean absolute error, and DSSIM being the structural dissimilarity which computes the perceptual dissimilarity between the output image and the ground truth image. The optimization is done using a stochastic gradient descent variant : the Adam optimizer.

2.3 SLAB-BY-SLAB INTEGRATION & LEVERAGING BVH

Since Gaussians basis functions have unbounded support, the authors enforce locality by introducing a threshold σ_ϵ below which a given entity's density σ_l is considered negligible and ignored. This approximation confines each Gaussian's influence to a local region and enables an efficient volume ray casting algorithm, reducing computation time. The scene is organized into a Bounding Volume Hierarchy (BVH), containing ellipsoids or spheres, which respectively corresponds to choosing elliptical or radial basis functions.

Once the BVH is constructed, rays are launched in parallel, and integration along each ray is performed sequentially, slab by slab. This slab-based approach accelerates rendering by focusing computation only on relevant regions, ie slabs that intersect the ray. It reduces unnecessary calculations, minimizes memory usage, and allows for early ray termination when contributions become negligible.

2.4 RESULTS

The authors tested their method on the Blender [3] and Mip-NeRF 360 [5] datasets. The experiments' results show the superior rendering quality of the RayGauss method compared to state of the art methods, for reasonable training and inference times.

3 OUR APPROACH

We are interested in improving the rendering pipeline used by RayGauss. To achieve this, we will explore two new approaches: modifying the basis functions used to define the color of the Gaussians and incorporating the handling of subtractive mixture for blending the colors of Gaussians.

3.1 BASIS FUNCTIONS

The functions chosen as basis for the density and colors by the initial paper are relevant from a mathematical point of view but are quite far from the usual rendering theory used in computer graphic. We are therefore interested in implementing a basis that is closer to the way a 3D object would be rendered using the traditional Phong illumination [2]:

$$C(x) = c_a a(x) + \sum_{i \in \text{lights}} n(\vec{x}) \cdot \vec{l}_i c_i d(x) + \sum_{i \in \text{lights}} (v(\vec{x}) \cdot r_i(\vec{x}))^\alpha c_i s(x)$$

where $C(x)$ is the observed color of point x ; $c(x)$, $a(x)$ and $r(x)$ its ambient, diffuse and specular absorption properties; $n(\vec{x})$ the normal of the object at x ; $\vec{l}_i$ the direction of the light from light source i ; $r_i(\vec{x})$ the reflected direction of the light from light source i on the object at x ; c_i the color of the light from light source i and $v(\vec{x})$ the view vector from the camera position to x .

The first and second components of the formula do not depend on the view direction and will therefore be a constant in our setting. The third term however is the one that the original spherical gaussians try to represent. Since their expression is quite different from the one of this term, we are interested in replacing spherical gaussians by functions closer to the third term, being of the form:

$$f_{a,b,\vec{d}}(\vec{v}) = \begin{cases} a \exp(b \log(|\vec{v} \cdot \vec{d}|)) & \text{if } \vec{v} \cdot \vec{d} \neq 0, \\ 0 & \text{if } \vec{v} \cdot \vec{d} = 0. \end{cases}$$

where $(a, c, \vec{d}) \in \mathbb{R} \times \mathbb{R} \times \mathbb{R}^3$ will be learned.

This function can be implemented in the cuda file `utils/sh_utils.cu` in the code by replacing the code of the spherical gaussians. However, it is also necessary to implement the backward function used for the gradient descent. For this we need the derivatives of f with respect to its variables. We have

$$\begin{aligned} \frac{\partial f_{a,b,\vec{d}}(v)}{\partial a} &= \begin{cases} \exp(b \log(|\vec{v} \cdot \vec{d}|)) & \text{if } \vec{v} \cdot \vec{d} \neq 0, \\ 0 & \text{if } \vec{v} \cdot \vec{d} = 0. \end{cases} \\ \frac{\partial f_{a,b,\vec{d}}(v)}{\partial b} &= \begin{cases} a \log(|\vec{v} \cdot \vec{d}|) \exp(b \log(|\vec{v} \cdot \vec{d}|)) & \text{if } \vec{v} \cdot \vec{d} \neq 0, \\ 0 & \text{if } \vec{v} \cdot \vec{d} = 0. \end{cases} \\ \frac{\partial f_{a,b,\vec{d}}(v)}{\partial d_i} &= \begin{cases} ab \frac{v_i}{\vec{v} \cdot \vec{d}} \exp(b \log(|\vec{v} \cdot \vec{d}|)) & \text{if } \vec{v} \cdot \vec{d} \neq 0, \\ 0 & \text{if } \vec{v} \cdot \vec{d} = 0. \end{cases} \\ \frac{\partial f_{a,b,\vec{d}}(v)}{\partial v_i} &= \begin{cases} ab \frac{d_i}{\vec{v} \cdot \vec{d}} \exp(b \log(|\vec{v} \cdot \vec{d}|)) & \text{if } \vec{v} \cdot \vec{d} \neq 0, \\ 0 & \text{if } \vec{v} \cdot \vec{d} = 0. \end{cases} \end{aligned}$$

where $x_i \in \mathbb{R}$ designates the i^{th} component of a vector $\vec{x} \in \mathbb{R}^3$.

3.2 COLOR BLENDING

The current implementation of RayGauss treats the color blending of the gaussians as the blending of emissive sources. It leverages the following approach:

- a ray is launched for every pixel, n positions are sampled along this ray, and all the intersected gaussians are detected (we will note IG the set of indices of these gaussians).
- for every position, the density d_i and color mc_i of the cloud of gaussians are stored in a buffer:

$$d_i = \sum_{k \in IG} \sigma_k w_{i,k} \quad \text{and} \quad mc_i = \sum_{k \in IG} c_k \sigma_k w_{i,k}$$

where c_k is the color of the gaussian k , σ_k its density and $w_{i,k}$ the blending weights computed using the position sampled and the position parameters of the gaussian k .

- The colors are then blended across all sampled positions to get the resulting ray color C :

$$C = \sum_{i=0}^n t_i \alpha_i mc_i \quad \text{with} \quad \alpha_i = 1 - \exp(-t d_i) \quad \text{and} \quad t_i = \prod_{j=0}^{i-1} (1 - \alpha_j)$$

where t is a constant. The order of the sampled positions is such that we are getting further from the camera when i increases. In practice, the computation of the sum is actually stopped when t_i is smaller than a fixed threshold, and the ray color is eventually blended with the color of the background, but we will neglect this for our reasoning.

This method does not allow to represent well translucent materials. Indeed, for such materials the blending should take into account the fact that certain colors are more absorbed than others. The color resulting from the superposition of two translucent materials is the product of their colors rather than their sum.

To account for this effect, we change the quantities we work with. We separate the effect of the gaussian densities σ and of the weights linked to the gaussian basis functions w . Both will be used to define with α to what extent light passes through a sampled point (and changes color accordingly, which will be parametrized by ct_i), but only the weights will be used with α' for the smooth blending of the gaussians.

Therefore, we define:

$$w_i = \sum_{k \in IG} w_{i,k} \quad ; \quad mc_i = \sum_{k \in IG} c_k w_{i,k}$$

To compute a ray color, we will use the formulas:

$$C = \sum_{i=0}^n ct_i t_i \alpha_i \frac{mc_i}{w_i} \quad \text{with} \quad ct_i = \prod_{j=0}^{i-1} (\alpha'_j \frac{mc_j}{w_j} + 1 - \alpha'_j) \quad ;$$

$$\alpha_i = 1 - \exp(-t d_i) \quad ; \quad \alpha'_i = 1 - \exp(-t w_i) \quad \text{and} \quad t_i = \prod_{j=0}^{i-1} (1 - \alpha_j)$$

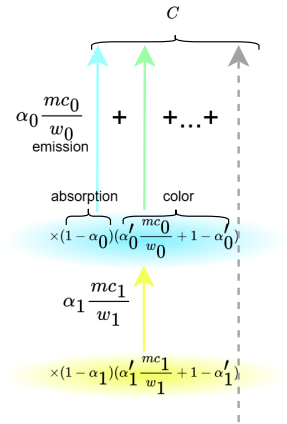


Figure 1: Diagram of the light behavior

It is necessary to implement these formulas (in `cuda_train\forward\gaussians_aabb.cu`) as well as their derivatives (in `cuda_train\backward\gaussians_aabb.cu`). We show here the main results, full computations can be found in the appendices.

$$\frac{\partial C}{\partial \sigma_k} = \sum_{i=0}^n w_{i,k} t \left[- \sum_{j=i}^n \left(ct_j t_j \alpha_j \frac{mc_j}{d_j} \right) + ct_i t_i \frac{mc_i}{d_i} \right]$$

$$\frac{\partial C}{\partial w_{i_0,k}} = \frac{\alpha'_{i_0} \frac{1}{w_{i_0}} (c_k - \frac{mc_{i_0}}{w_{i_0}}) + (\frac{mc_{i_0}}{w_{i_0}} - 1)t(1 - \alpha'_{i_0})}{\alpha'_{i_0} \frac{mc_{i_0}}{w_{i_0}} + 1 - \alpha'_{i_0}} \left(\sum_{i=i_0+1}^n ct_i t_i \alpha_i \frac{mc_i}{w_i} \right) - t\sigma_k \left(\sum_{i=i_0}^n ct_i t_i \alpha_i \alpha'_i \frac{mc_i}{w_i} \right)$$

$$+ ct_{i_0} t_{i_0} t\sigma_k \frac{mc_{i_0}}{w_{i_0}} + ct_{i_0} t_{i_0} \alpha_{i_0} \frac{1}{w_{i_0}} (c_k - \frac{mc_{i_0}}{w_{i_0}})$$

And:

$$\frac{\partial C}{\partial c_k} = \sum_{i=0}^n \left(\frac{\alpha'_i \frac{w_{i,k}}{w_i}}{\alpha'_i \frac{mc_i}{w_i} + 1 - \alpha'_i} \left(\sum_{j=i+1}^n ct_j t_j \alpha_j \frac{mc_j}{w_j} \right) + ct_i t_i \alpha_i \frac{w_{i,k}}{w_i} \right)$$

where $k \in IG$ and $i_0 \in \llbracket 0, n \rrbracket$. All formulas hold if the $\alpha'_i mc_i + 1 - \alpha'_i$ and w_i are not equal to zero. If they are, the associated terms are equal to zero.

4 RESULTS

Due to the high computation times involved with RayGauss, we were only able to get relevant results for the Lego scene of the Blender dataset [3]. We name "Spec" a model with modified basis functions, and "Transp" a model that incorporates our modified color blending. "SH" indicates the degree of the spherical harmonics, "SG" the one of the spherical gaussians, and M the one of our modified functions.

Method	PSNR
RayGauss (SH2,SG1)	37.10
Spec (M1)	37.03
Spec (M2)	37.06
Spec (M3)	36.90
Spec (SH2, M1)	37.14
Transp & Spec (SH2, M1)	37.19

Table 1: PSNR comparison of different methods on the Lego scene

These results highlight the importance of the spherical harmonic for this approach. They are necessary to learn the low frequency variations of the gaussian colors and, when alone, our other basis functions cannot compensate the lack of spherical harmonics. It even seems that adding too many high frequency basis functions leads to over-fitting and to a worse PSNR.

The results also indicate that better performance can be achieved by using our custom basis functions in conjunction with spherical harmonics, and that our modified color blending approach further enhances the results.

To obtain more reliable results, it would be necessary to conduct these experiments on all scenes from the Blender dataset. However, this was not feasible due to the training times, which are approximately four hours on our machines, regardless of the method used.

Here are some visualizations of the gaussian cloud produced by the Transp & Spec approach on the Lego scene:



Figure 2: Visualization of the results

Visually, the main difference between these generated images and the ground truth ones is often the lack of specular reflections. The images still are of very good quality.

5 CONCLUSION

In conclusion, the RayGauss paper presents an innovative approach to photorealistic novel view synthesis (NVS) by combining the strengths of volumetric ray casting with locally adaptive Gaussian basis functions. One of its main contributions is the physically coherent formulation of radiance c and density σ fields using anisotropic Gaussians with direction-dependent emission modeled by a combination of spherical harmonics and spherical Gaussians. This setup allows for accurate representation of both scene geometry and view-dependent lighting effects.

Another significant contribution is the development of an efficient slab-by-slab ray casting algorithm for irregularly spaced Gaussians, which was previously a major challenge in the field. By leveraging a GPU-accelerated BVH structure via NVIDIA OptiX, the method achieves interactive inference speeds while avoiding common artifacts of splatting, such as flickering and inaccurate occlusion handling.

The paper provides comprehensive experimental validation across standard datasets (Blender, Mip-NeRF360) and clearly outperforms prior methods in terms of rendering quality (PSNR, SSIM, LPIPS), while maintaining practical training times. Importantly, it demonstrates that accurate ray casting, despite being computationally more demanding than splatting, can achieve real-time performance with superior visual results when properly optimized.

However, some limitations remain. The training time, while reasonable, is higher than splatting methods, and memory usage could become a bottleneck for extremely large scenes due to the BVH and slab buffer requirements. Additionally, while the paper emphasizes the improved rendering quality, it could have further analyzed failure cases or scenes where splatting might still be preferable. Finally, there is limited discussion on generalization across scene types or scalability to dynamic scenes.

To build upon this work, we proposed two new approaches. The first involves using alternative basis functions, more closely aligned with Phong illumination theory, to replace spherical Gaussians. The second focuses on handling subtractive color blending for translucent materials. Experimental results showed that both modifications improved reconstruction performance. However, since these experiments were somewhat limited in scope, it would be valuable to confirm our findings by applying our approach to the reconstruction of other types of scenes.

REFERENCES

- [1] Hugo Blanc, Jean-Emmanuel Deschaud, and Alexis Paljic. *RayGauss: Volumetric Gaussian-Based Ray Casting for Photorealistic Novel View Synthesis*. 2025. arXiv: 2408.03356 [cs.CV]. URL: <https://arxiv.org/abs/2408.03356>.
- [2] Bui Tuong Phong. “Illumination for computer generated pictures”. In: *Commun. ACM* 18.6 (June 1975), pp. 311–317. ISSN: 0001-0782. DOI: 10.1145/360825.360839. URL: <https://doi.org/10.1145/360825.360839>.
- [3] Ben Mildenhall et al. “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis”. In: *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I*. Glasgow, United Kingdom: Springer-Verlag, 2020, pp. 405–421. ISBN: 978-3-030-58451-1. DOI: 10.1007/978-3-030-58452-8_24. URL: https://doi.org/10.1007/978-3-030-58452-8_24.
- [4] Bernhard Kerbl et al. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. In: *ACM Trans. Graph.* 42.4 (July 2023). ISSN: 0730-0301. DOI: 10.1145/3592433. URL: <https://doi.org/10.1145/3592433>.
- [5] Jonathan T. Barron et al. *Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields*. 2022. arXiv: 2111.12077 [cs.CV]. URL: <https://arxiv.org/abs/2111.12077>.

6 APPENDIX

6.1 COMPUTATION OF DERIVATIVES FOR COLOR BLENDING

First, here are some useful derivatives for our computations:

$$\frac{\partial mc_i/w_i}{\partial \sigma_k} = 0 \quad ; \quad \frac{\partial \alpha'_i}{\partial \sigma_k} = 0 \quad \text{and} \quad \frac{\partial \alpha_i}{\partial \sigma_k} = t \frac{w_{i,k}}{w_i} \exp(-td_i/w_i) = t \frac{w_{i,k}}{w_i} (1 - \alpha_i)$$

$$\frac{\partial mc_i/w_i}{\partial w_{i_0,k}} = \mathbb{1}_{i=i_0} \frac{c_k w_i - mc_i}{w_i^2} = \mathbb{1}_{i=i_0} \frac{1}{w_i} (c_k - \frac{mc_i}{w_i}) \quad \text{and} \quad \frac{\partial \alpha_i}{\partial w_{i_0,k}} = \mathbb{1}_{i=i_0} t \sigma_k \exp(-td_i) = \mathbb{1}_{i=i_0} t \sigma_k (1 - \alpha_i)$$

$$\frac{\partial \alpha'_i}{\partial w_{i_0,k}} = \mathbb{1}_{i=i_0} t \exp(-tw_i) = \mathbb{1}_{i=i_0} t (1 - \alpha'_i)$$

$$\frac{\partial mc_i/w_i}{\partial c_k} = \frac{w_{i,k}}{w_i} \quad ; \quad \frac{\partial \alpha'_i}{\partial c_k} = 0 \quad \text{and} \quad \frac{\partial \alpha_i}{\partial c_k} = 0$$

We can now compute the derivatives we are interested in:

$$\begin{aligned} \frac{\partial C}{\partial \sigma_k} &= \sum_{i=0}^n ct_i \frac{\partial t_i}{\partial \sigma_k} \alpha_i \frac{mc_i}{d_i} + \sum_{i=0}^n ct_i t_i \frac{\partial \alpha_i}{\partial \sigma_k} \frac{mc_i}{d_i} \\ &= - \sum_{i=0}^n ct_i \left(\sum_{j=0}^{i-1} \prod_{l \neq j} (1 - \alpha_l) \frac{\partial \alpha_j}{\partial \sigma_k} \right) \alpha_i \frac{mc_i}{d_i} + \sum_{i=0}^n ct_i t_i \frac{\partial \alpha_i}{\partial \sigma_k} \frac{mc_i}{d_i} \\ &= - \sum_{i=0}^n ct_i t_i \left(\sum_{j=0}^{i-1} w_{j,k} t \right) \alpha_i \frac{mc_i}{d_i} + \sum_{i=0}^n ct_i t_i w_{i,k} t (1 - \alpha_i) \frac{mc_i}{d_i} \\ &= - \sum_{j=0}^n w_{j,k} t \left(\sum_{i=j}^n ct_i t_i \alpha_i \frac{mc_i}{d_i} \right) + \sum_{i=0}^n ct_i t_i w_{i,k} t \frac{mc_i}{d_i} \\ &= \sum_{i=0}^n w_{i,k} \left(-t \sum_{j=i}^n ct_j t_j \alpha_j \frac{mc_j}{d_j} + ct_i t_i t \frac{mc_i}{d_i} \right) \end{aligned}$$

$$\begin{aligned}
\frac{\partial C}{\partial w_{i_0,k}} &= \sum_{i=i_0+1}^n \left(\prod_{l \neq i_0} (\alpha'_l \frac{mc_l}{w_l} + 1 - \alpha'_l) \left(\frac{mc_{i_0}}{w_{i_0}} \frac{\partial \alpha'_{i_0}}{\partial w_{i_0,k}} + \alpha'_{i_0} \frac{\partial mc_{i_0}/w_{i_0}}{\partial w_{i_0,k}} - \frac{\partial \alpha'_{i_0}}{\partial w_{i_0,k}} \right) \right) t_i \alpha_i \frac{mc_i}{w_i} \\
&\quad - \sum_{i=i_0+1}^n ct_i \left(\prod_{l \neq i_0} (1 - \alpha_l) \frac{\partial \alpha_{i_0}}{\partial w_{i_0,k}} \right) \alpha_i \frac{mc_i}{w_i} + ct_{i_0} t_{i_0} \frac{\partial \alpha_{i_0}}{\partial w_{i_0,k}} \frac{mc_{i_0}}{w_{i_0}} + ct_{i_0} t_{i_0} \alpha_{i_0} \frac{1}{w_{i_0}} (c_k - \frac{mc_{i_0}}{w_{i_0}}) \\
&= \sum_{i=i_0+1}^n ct_i \left(\frac{\alpha'_{i_0} \frac{1}{w_{i_0}} (c_k - \frac{mc_{i_0}}{w_{i_0}}) + (\frac{mc_{i_0}}{w_{i_0}} - 1) t (1 - \alpha'_{i_0})}{\alpha'_{i_0} \frac{mc_{i_0}}{w_{i_0}} + 1 - \alpha'_{i_0}} \right) t_i \alpha_i \frac{mc_i}{w_i} - \sum_{i=i_0+1}^n ct_i t_i t_{\sigma_k} \alpha_i \frac{mc_i}{w_i} \\
&\quad + ct_{i_0} t_{i_0} t_{\sigma_k} (1 - \alpha_{i_0}) \frac{mc_{i_0}}{w_{i_0}} + ct_{i_0} t_{i_0} \alpha_{i_0} \frac{1}{w_{i_0}} (c_k - \frac{mc_{i_0}}{w_{i_0}}) \\
&= \frac{\alpha'_{i_0} \frac{1}{w_{i_0}} (c_k - \frac{mc_{i_0}}{w_{i_0}}) + (\frac{mc_{i_0}}{w_{i_0}} - 1) t (1 - \alpha'_{i_0})}{\alpha'_{i_0} \frac{mc_{i_0}}{w_{i_0}} + 1 - \alpha'_{i_0}} \left(\sum_{i=i_0+1}^n ct_i t_i \alpha_i \frac{mc_i}{w_i} \right) - t_{\sigma_k} \left(\sum_{i=i_0}^n ct_i t_i \alpha_i \alpha'_i \frac{mc_i}{w_i} \right) \\
&\quad + ct_{i_0} t_{i_0} t_{\sigma_k} \frac{mc_{i_0}}{w_{i_0}} + ct_{i_0} t_{i_0} \alpha_{i_0} \frac{1}{w_{i_0}} (c_k - \frac{mc_{i_0}}{w_{i_0}}) \\
\frac{\partial C}{\partial c_k} &= \sum_{i=0}^n \left(\sum_{j=0}^{i-1} \prod_{l \neq j} (\alpha'_l \frac{mc_l}{w_l} + 1 - \alpha'_l) \alpha'_j \frac{\partial mc_j/w_j}{\partial c_k} \right) t_i \alpha_i \frac{mc_i}{w_i} + \sum_{i=0}^n ct_i t_i \alpha_i \frac{w_{i,k}}{w_i} \\
&= \sum_{i=0}^n \left(\frac{\alpha'_i \frac{w_{i,k}}{w_i}}{\alpha'_i \frac{mc_i}{w_i} + 1 - \alpha'_i} \left(\sum_{j=i+1}^n ct_j t_j \alpha_j \frac{mc_j}{w_j} \right) + ct_i t_i \alpha_i \frac{w_{i,k}}{w_i} \right)
\end{aligned}$$